SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA-1410 COMPILER DESIGN – UNIT I

ASSIGNMENT 1 – CO1

Development of a Domain-Specific Compiler Front-End using LEX for Federated Cyber Threat Detection in Security Operations Centers

Student Details	
Student Name	Ch.Venkata Ramayogi
Register Number	192424319
Department / Section	B.E-AIDS/SLOT-D
Academic Year / Semester	SEP–DEC 2026
Faculty Name	Dr. N. Deepa
Total Marks	50

Submitted in partial fulfilment of the requirements for Compiler Design – Unit I.

CONTENTS

- Task 1 – Language Processors
- Task 2 – Phases of a Compiler
- Task 3 – Grouping of Compiler Phases
- Task 4 – Compiler Construction Tools
- Task 5 – Role of the Lexical Analyzer
- Task 6 – Specification of Tokens
- Task 7 – Recognition of Tokens
- Task 8 – LEX Specification
- Task 9 – Implementation and Testing
- Task 10 – Scanner Performance
- Task 11 – Final Design Recommendation

- Task 12A – Impact Analysis
- Task 12B – Evaluation of the Original Design
- Task 12C – Adaptation / Redesign
- Task 12D – Evidence-Based Defence
- Conclusion
- Appendix A – Flex Program
- Appendix B – Test Data and Commands
- References

Assignment scope: This work designs a lexical front-end for the ThreatRule domain-specific language. The scanner recognises lexical units, records identifiers, validates selected token patterns and reports recoverable lexical errors. Full grammatical parsing, semantic analysis and code generation are later compiler phas

TASK 1 – LANGUAGE PROCESSORS (4 MARKS)

A language processor is system software that translates or executes a program written in one language into another form that a computer can use. ThreatRule rules are created by branch Security Operations Centres (SOCs). A central SOC needs a consistent way to validate these rules before deployment.

Processor	**Working method**	**Advantages**	**Limitations / suitability**
Compiler	Translates the complete source program into target or intermediate code before execution.	Fast repeated execution; detects errors before deployment; reusable output.	Compilation is needed after source changes. Best choice for batch validation of ThreatRule files.
Interpreter	Reads, translates and executes statements during runtime.	Immediate feedback; useful for interactive testing.	Repeated interpretation may be slower. Execution and validation are mixed.
Assembler	Converts assembly-language mnemonics into machine instructions.	Produces low-level machine code and handles hardware-specific instructions.	ThreatRule is a high-level domain language, not assembly language.

Hybrid processor	Compiles source into bytecode/intermediate form and uses a virtual machine or JIT compiler.	Portable intermediate form; can combine flexibility and optimisation.	Adds a runtime/VM layer that is unnecessary for a small rule-validation tool.

1.1 Supporting language-processing components

- Preprocessor: handles source text before compilation, such as file inclusion, macro expansion and conditional compilation. ThreatRule could later use it to include common rule libraries.
- Linker: combines object modules and libraries, resolves external references and produces an executable. It is useful when the scanner, parser and support functions are separate files.
- Loader: places the executable in memory, prepares its runtime environment and starts execution.

1.2 Selected processor and justification

A compiler is selected for the central SOC. Incoming rule files can be checked in batches, lexical and syntax errors can be detected before deployment, and the generated scanner can be reused by all branch offices. A common compiler front-end ensures that every branch follows the same token definitions.

1.3 Language-processing system

```
ThreatRule source files from branch SOCs
                |
                v
        Preprocessing (optional)
                |
                v
        Compiler front-end
        [LEX scanner + parser]
                |
                v
        Semantic validation
                |
                v
        Intermediate representation
                |
```

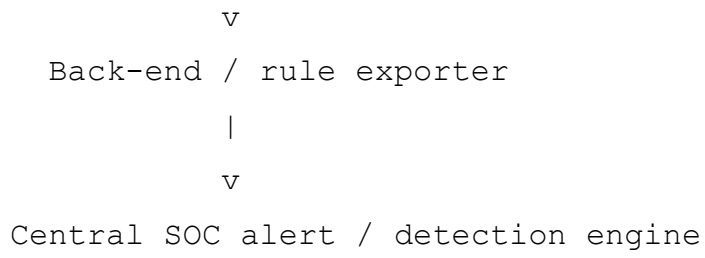


Figure 1. ThreatRule language-processing system.

TASK 2 – PHASES OF A COMPILER (4 MARKS)

A compiler is divided into phases. Each phase performs a specific operation and passes its result to the next phase. The symbol table and error handler support several phases.

Phase	Main function	ThreatRule example / output
Lexical analysis	Groups characters into lexemes and produces tokens.	RULE, IF, ID, IP_ADDRESS, INTEGER, AND, THEN.
Syntax analysis	Checks whether the token sequence follows the grammar.	Checks whether RULE name IF condition THEN action is correctly structured.
Semantic analysis	Checks meaning, declarations, types and domain constraints.	Checks that an IP field contains an IP token and that a rule has a valid action.
Intermediate code generation	Converts the validated source into an intermediate representation (IR).	Creates an internal rule record or condition tree.
Code optimisation	Improves the IR without changing its meaning.	Removes duplicate conditions and simplifies Boolean expressions.
Target-code generation	Produces output for the selected target platform.	Exports a rule for the central SOC detection engine.

2.1 Supporting activities

The symbol table stores information about identifiers, such as their names, token category, scope and other attributes. The error handler reports lexical, syntax and semantic errors with useful locations.

These activities interact with multiple compiler phases rather than operating as independent translation phases.

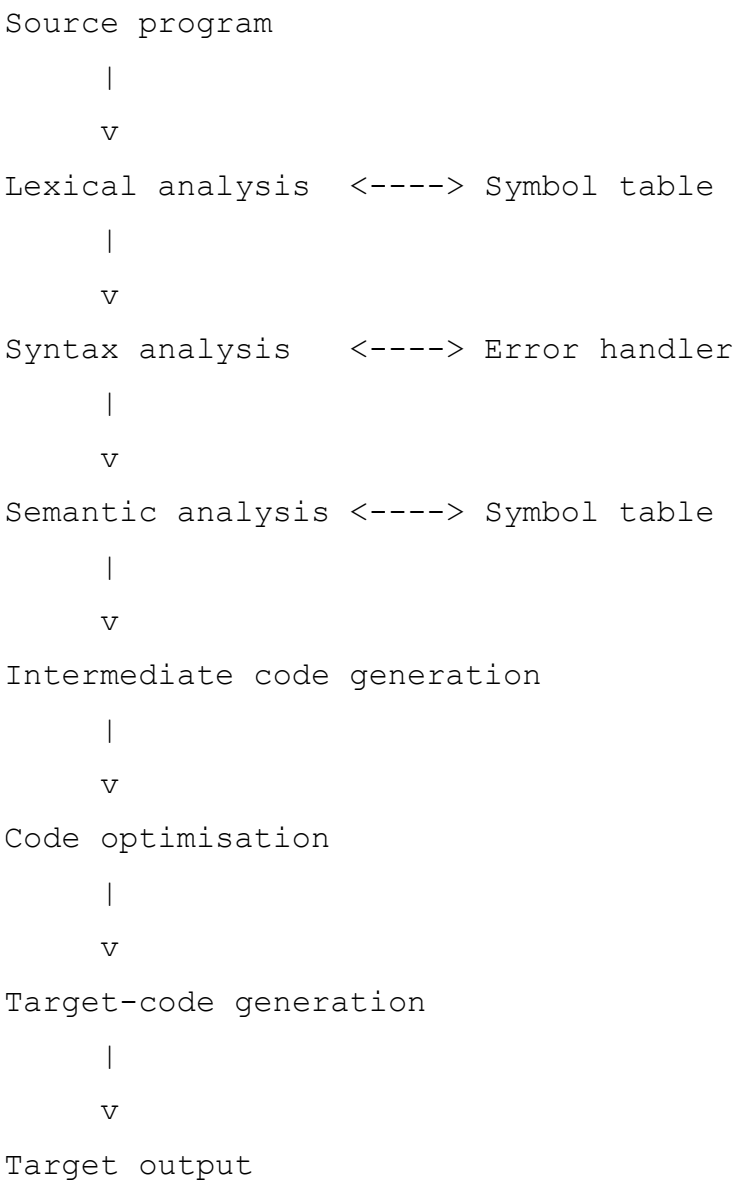
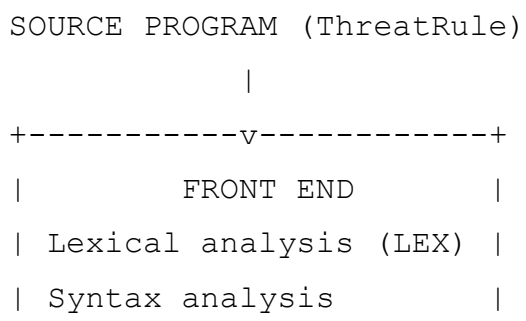


Figure 2. Main compiler phases and supporting activities.

TASK 3 – GROUPING OF COMPILER PHASES (3 MARKS)



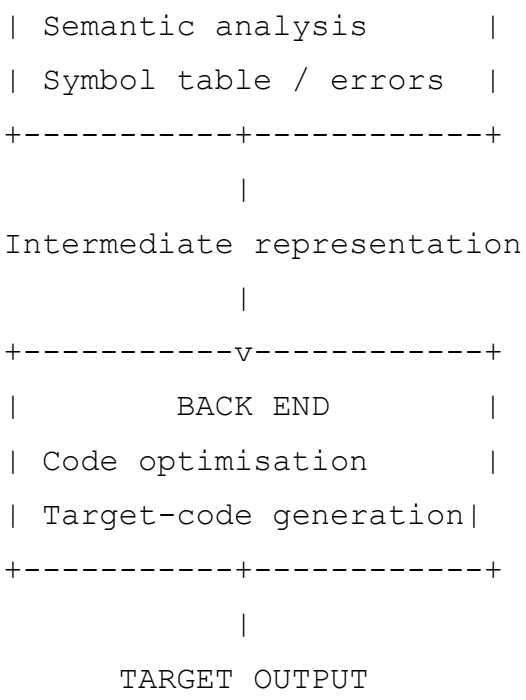


Figure 3. Front-end and back-end grouping.

The front end analyses the source language and produces an intermediate representation. It mainly depends on ThreatRule syntax and meaning. The back end transforms the intermediate representation into code for a selected target runtime or platform.

A common front-end helps branch SOCs because every branch uses the same token rules, grammar and semantic checks, even if branches use different alert engines or output formats. Only the back end needs to change for a different target.

A pass is one complete traversal of the source program or an intermediate representation to perform defined operations. A one-pass design can be simple and fast, but dependencies between parsing, type checking and code generation can make it difficult. A two-or-more-pass design is recommended: Pass 1 scans/parses and builds symbol information; Pass 2 performs semantic validation and emits the selected intermediate form. This improves diagnostics and makes future extensions easier.

TASK 4 – COMPILER CONSTRUCTION TOOLS (3 MARKS)

Tool category	**Purpose**	**Placement in ThreatRule compiler**
Scanner generator	Generates a lexical analyser from regular-expression rules.	LEX/Flex generates the scanner from threatrule.l.

Parser generator	Generates a parser from a context-free grammar.	YACC/Bison can generate the ThreatRule parser.
Syntax-directed translation engine	Attaches semantic actions to grammar productions.	Builds syntax-tree nodes and intermediate instructions during parsing.
Code-generator generator	Helps create target-code generators from machine descriptions.	Can support output for different SOC rule runtimes.
Data-flow analysis engine	Analyses definitions, uses, control flow and optimisation opportunities.	Supports back-end optimisation of intermediate code.
Compiler-construction toolkit	Provides reusable structures and utilities.	Provides symbol-table, syntax-tree, diagnostics and IR support.

LEX/Flex is preferred to a fully hand-written scanner because token patterns are written declaratively as regular expressions. Generated code handles matching efficiently, rules are easier to extend, and the same specification can be regenerated consistently. A hand-written scanner offers fine control but usually needs more maintenance and careful handling of edge cases.

TASK 5 – ROLE OF THE LEXICAL ANALYZER (4 MARKS)

The lexical analyser is the first compiler phase. It reads the source character stream, groups characters into lexemes, and produces tokens for the parser. It skips irrelevant whitespace and comments, tracks line numbers and reports invalid character sequences.

A token is a category such as ID or INTEGER. A pattern describes all lexemes belonging to a token. A lexeme is the actual character sequence, such as attempts or 22.

Token	**Pattern idea**	**ThreatRule lexeme**
KEYWORD	One of the reserved words.	RULE, IF, THEN, AND
ID	Letter or underscore followed by letters, digits or underscores.	ssh_attack
IP_ADDRESS	Four decimal octets separated by dots; each octet is 0–255.	10.0.0.5

INTEGER	One or more decimal digits.	22
STRING	Text enclosed in double quotes.	"Repeated login attempts"
HASH	Exactly 64 hexadecimal characters.	SHA-256 digest
CVE	CVE-year-number format.	CVE-2024-12345
URL	A supported URL beginning with http, https or ftp.	https://example.org/rule

5.1 Input-to-token example

```
RULE ssh_attack IF src_ip == 10.0.0.5 AND attempts > 5
THEN ALERT "Repeated login attempts";
```

The scanner returns a sequence similar to: RULE, ID(ssh_attack), IF, ID(src_ip), EQ, IP_ADDRESS(10.0.0.5), AND, ID(attempts), GT, INTEGER(5), THEN, ALERT, STRING, SEMI. The parser receives tokens rather than individual characters.

TASK 6 – SPECIFICATION OF TOKENS (4 MARKS)

Regular expressions describe token patterns. The following definitions form a practical lexical specification. The scanner should check reserved words before returning a general identifier.

Token	**Regular expression / rule**	**Explanation**
LETTER	[A-Za-z]	An English alphabetic character.
DIGIT	[0-9]	A decimal digit.
ID	[A-Za-z_][A-Za-z0-9_]*	Starts with a letter or underscore.
INTEGER	[0-9]+	One or more digits.
IP_ADDRESS	([0-9]{1,3}\.){3}[0-9]{1,3}	Shape of IPv4; octet range 0–255 must also be checked.
STRING	\"([^\"\\\\]|\\\\.)*\"	Quoted string with escaped characters.
COMMENT	//[^\n]*	Single-line comment.

HASH	[A-Fa-f0-9]{64}	64 hexadecimal digits for SHA-256.
CVE	CVE-[0-9]{4}-[0-9]{4,}	CVE identifier shape.
URL	(https?|ftp)://[^ \t\r\n;\"<>]+	URL until whitespace or selected delimiters.
EQ / NE	== / !=	Equality and inequality operators.
LE / GE	<= / >=	Less-than-or-equal and greater-than-or-equal.

Important validation note: a regular expression that checks the shape of an IPv4 address may still match values such as 999.1.1.1. The scanner must convert each octet to an integer and verify that it is between 0 and 255. Similarly, a CVE pattern validates its general format, not whether the identifier exists in an official vulnerability database.

TASK 7 – RECOGNITION OF TOKENS (4 MARKS)

A finite automaton recognises a token by moving between states as input characters are read. A final or accepting state indicates that a valid token has been recognised. The following simplified state diagrams illustrate common token classes.

7.1 Identifier recognition

```
Start --letter or '_'--> S1
S1    --letter/digit/'_'--> S1
S1    --other character--> ACCEPT ID (retract other character)
```

An identifier must begin with a letter or underscore. After the first character, letters, digits and underscores are accepted. Example: _alert2.

7.2 Integer recognition

```
Start --digit--> S1
S1    --digit--> S1
S1    --non-digit--> ACCEPT INTEGER
```

One or more digits form an integer. Example: 507. A sign is treated as a separate operator unless the language specification explicitly defines signed integer literals.

7.3 Relational operator recognition

```
 Start -- '<' --> S1 -- '=' --> ACCEPT LE (<=)
                    -- other --> ACCEPT LT (<), retract
 Start -- '>' --> S2 -- '=' --> ACCEPT GE (>=)
                    -- other --> ACCEPT GT (>), retract
 Start -- '=' --> S3 -- '=' --> ACCEPT EQ (==)
                    -- other --> report invalid '=' if assignment is
not allowed
 Start -- '!' --> S4 -- '=' --> ACCEPT NE (!=)
                    -- other --> report invalid '!'
```

The scanner uses look-ahead to distinguish one-character and two-character operators.

7.4 IPv4 address recognition

```
 Start -> read 1 to 3 digits -> dot
       -> read 1 to 3 digits -> dot
       -> read 1 to 3 digits -> dot
       -> read 1 to 3 digits -> ACCEPT candidate
       -> validate each numeric octet is 0..255
```

Example 192.168.1.10 is valid. Example 192.168.1.300 has the correct general shape but is rejected because the last octet is greater than 255.

7.5 Token recognition examples

Input	Expected result	Reason
server_01	ID	Valid identifier.
45	INTEGER	Digits only.
10.1.2.3	IP_ADDRESS	Four octets in valid range.
10.1.2.300	LEXICAL ERROR	Octet 300 is outside 0–255.
==	EQ	Two-character equality operator.
@	LEXICAL ERROR	Not a defined ThreatRule character.

TASK 8 – LEX SPECIFICATION (5 MARKS)

LEX/Flex accepts definitions, regular-expression rules and C actions. The definitions section contains declarations and helper functions. The rules section maps patterns to actions. The final section may contain a main function or additional C code.

```
%option noyywrap yylineno
%x COMMENT

%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static int errors = 0;
static int tokens = 0;

static void emit(const char *name, const char *text) {
    printf("LINE %-4d TOKEN %-12s LEXEME %s\n",
           yylineno, name, text);
    tokens++;
}
static void lexical_error(const char *msg) {
    fprintf(stderr, "LINE %d ERROR: %s near '%s'\n",
            yylineno, msg, yytext);
    errors++;
}
%}

LETTER      [A-Za-z]
DIGIT       [0-9]
ID          [A-Za-z_][A-Za-z0-9_]*
INT         {DIGIT}+
IP          {DIGIT}{1,3}(\.{DIGIT}{1,3}){3}
HASH        [A-Fa-f0-9]{64}
```

```
CVE         CVE-[0-9]{4}-[0-9]{4,}
URL         (https?|ftp)://[^ \t\r\n;\"<>]+
STRING      \"([^\\"\\\\]|\\\\.)*\"

%%
"RULE"      { emit("RULE", yytext); }
"IF"        { emit("IF", yytext); }
"AND"       { emit("AND", yytext); }
"OR"        { emit("OR", yytext); }
"THEN"      { emit("THEN", yytext); }
"ALERT"     { emit("ALERT", yytext); }
"BLOCK"     { emit("BLOCK", yytext); }

"=="        { emit("EQ", yytext); }
"!="        { emit("NE", yytext); }
"<="        { emit("LE", yytext); }
">="        { emit("GE", yytext); }
"<"         { emit("LT", yytext); }
">"         { emit("GT", yytext); }
"="         { emit("ASSIGN", yytext); }
";"         { emit("SEMICOLON", yytext); }
"("         { emit("LPAREN", yytext); }
")"         { emit("RPAREN", yytext); }

{URL}       { emit("URL", yytext); }
{CVE}       { emit("CVE", yytext); }
{HASH}      { emit("SHA256_HASH", yytext); }
{STRING}    { emit("STRING", yytext); }
{IP}        { emit("IP_CANDIDATE", yytext); }
{INT}       { emit("INTEGER", yytext); }

"//"[^\n]* { /* ignore single-line comment */ }
[ \t\r]+  { /* ignore horizontal whitespace */ }
\n         { /* yylineno is maintained by Flex */ }
```

```
{ID}        { emit("IDENTIFIER", yytext); }

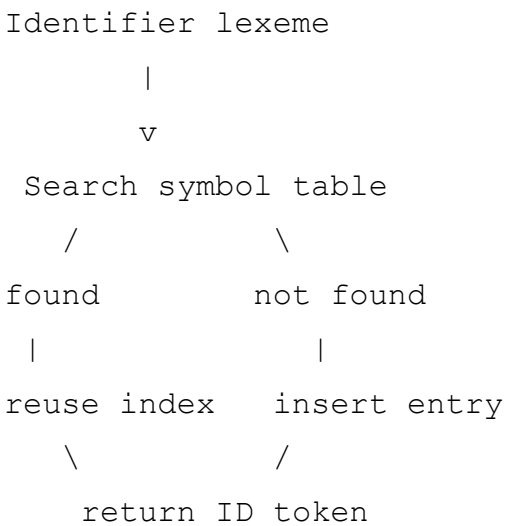
.           { lexical_error("unexpected character"); }

%%
int main(void) {
    yylex();
    printf("\nTotal tokens: %d\nLexical errors: %d\n",
           tokens, errors);
    return errors ? 1 : 0;
}
```

Note: This teaching scanner recognises an IPv4-shaped candidate. A production implementation should validate each octet numerically in a helper function before emitting IP_ADDRESS. It should also add a dedicated rule for unterminated strings and block comments if those constructs are part of the language. Flex's longest-match rule applies; when two rules match the same length, the earlier rule wins. Therefore keywords and special literals should be placed before the general identifier rule.

8.1 Symbol-table concept

When an identifier is recognised, a production scanner or parser can look it up in a symbol table. If the name is new, it is inserted; otherwise, the existing entry is reused. A simple table can store the identifier string, token category, scope and an index. In a larger implementation, a hash table is preferred for faster lookup.



TASK 9 – IMPLEMENTATION AND TESTING (5 MARKS)

9.1 Build procedure

1. Install Flex and a C compiler such as GCC.
2. Save the scanner specification as threatrule.l.
3. Generate C source using: flex threatrule.l
4. Compile the generated file using: gcc lex.yy.c -o threatrule
5. Run the scanner using: ./threatrule < sample.tr (on Windows, use the corresponding executable name and input redirection).
6. Compare the output with the expected token sequence and check that invalid characters are reported with line numbers.

9.2 Test input

```
RULE ssh_attack IF src_ip == 10.0.0.5 AND attempts > 5
THEN ALERT "Repeated login attempts";

RULE block_bad_host IF src_ip == 10.1.2.300
THEN BLOCK;
```

9.3 Test cases

Test	Input / condition	Expected result
T1	RULE keyword	Recognised as RULE, not IDENTIFIER.
T2	ssh_attack	Recognised as IDENTIFIER.
T3	attempts >= 5	IDENTIFIER, GE, INTEGER.
T4	10.0.0.5	Recognised as IPv4 after octet validation.
T5	10.0.0.300	Rejected as invalid IPv4 address.
T6	64 hexadecimal characters	Recognised as SHA256_HASH.
T7	CVE-2024-12345	Recognised as CVE.
T8	https://example.org/a	Recognised as URL.

T9	@	Reports unexpected character and continues.
T10	// comment	Comment is ignored.

Expected behaviour: valid input produces a sequence of tokens. Invalid input produces a diagnostic containing the line number and offending lexeme. The scanner should continue after a recoverable invalid character so that one error does not hide later errors.

9.4 Sample output format

```
LINE 1    TOKEN RULE          LEXEME RULE
LINE 1    TOKEN IDENTIFIER    LEXEME ssh_attack
LINE 1    TOKEN IF            LEXEME IF
LINE 1    TOKEN IDENTIFIER    LEXEME src_ip
LINE 1    TOKEN EQ            LEXEME ==
LINE 1    TOKEN IP_CANDIDATE  LEXEME 10.0.0.5
LINE 1    TOKEN AND           LEXEME AND
LINE 1    TOKEN IDENTIFIER    LEXEME attempts
LINE 1    TOKEN GT            LEXEME >
LINE 1    TOKEN INTEGER       LEXEME 5
LINE 2    TOKEN THEN          LEXEME THEN
LINE 2    TOKEN ALERT         LEXEME ALERT
LINE 2    TOKEN STRING        LEXEME "Repeated login attempts"
LINE 2    TOKEN SEMICOLON     LEXEME ;
```

This is an illustrative output format. Exact line numbers and token labels depend on the final scanner implementation and the input file. The sample output is not a claim that the code has been executed in the user's environment.

TASK 10 – SCANNER PERFORMANCE (4 MARKS)

Scanner performance is important because a central SOC may receive rule files from many branch offices. The scanner should process large files quickly, use memory efficiently and report errors without stopping unnecessarily.

Metric	Meaning	How to measure

Throughput	Number of input bytes or rules processed per second.	File size divided by scan time; repeat trials and report median.
Latency	Time required to begin producing the first token or diagnostic.	Measure from scanner start to first output.
Memory use	Memory occupied by the scanner and symbol table.	Use an operating-system process monitor or profiling tool.
Error recovery	Ability to continue scanning after malformed input.	Count valid tokens found after deliberately inserted errors.
Scalability	How performance changes as input size increases.	Test small, medium and large files with the same machine.

For a fair experiment, use the same computer, compiler options and test corpus. Run each file several times, discard unusual startup effects if justified, and report the median. Do not invent scan times or token counts. Actual measured values should be entered after running the program.

Input size	**File size**	**Run 1 (s)**	**Run 2 (s)**	**Run 3 (s)**	**Median (s)**	**Throughput**
Small	_____	_____	_____	_____	_____	_____
Medium	_____	_____	_____	_____	_____	_____
Large	_____	_____	_____	_____	_____	_____

Throughput can be calculated as: throughput = input file size in bytes / median scan time in seconds. If rule count is used, state that unit clearly (rules/second).

TASK 11 – FINAL DESIGN RECOMMENDATION (3 MARKS)

The recommended design is a Flex-generated lexical analyser connected to a parser generated by Bison/YACC. The scanner should recognise ThreatRule keywords, identifiers, integers, strings, operators, IPv4 candidates, SHA-256 hashes, CVE identifiers and URLs. It should maintain line numbers, skip comments and whitespace, and report recoverable lexical errors.

- Use a shared token specification for all branch SOCs to avoid inconsistent rule interpretation.
- Keep lexical analysis separate from syntax and semantic validation.

- Validate IPv4 octet ranges and other domain-specific constraints in helper functions or semantic analysis.
- Use a symbol table for identifiers and provide line-numbered diagnostics.
- Benchmark with realistic large files before central deployment.
- Keep the scanner modular so new threat-indicator formats can be added without rewriting the entire compiler.

This approach provides consistency, maintainability and a clear path for later parser and back-end development.

TASK 12A – IMPACT ANALYSIS (2 MARKS)

The original scanner is suitable for basic language tokens but a federated cyber-threat system also needs to recognise threat indicators such as SHA-256 hashes, CVE identifiers and URLs. If these values are treated as ordinary identifiers or split into several tokens, the parser may receive incorrect input and the SOC may reject or misinterpret a rule.

Area	**Impact of missing token support**	**Required improvement**
Threat coverage	Hash, CVE and URL values may be split or rejected.	Add dedicated regular-expression rules.
Accuracy	A malformed value may be accepted as a valid token.	Validate format and applicable numeric ranges.
Diagnostics	A scanner that stops at the first error hides later issues.	Report line number and continue after recoverable errors.
Maintainability	Duplicated token definitions across branches create inconsistent behaviour.	Maintain one shared scanner specification.
Performance	More rules can increase matching work.	Measure throughput on representative large files.

The changes improve the quality of input validation, but lexical recognition alone does not prove that a URL is safe or that a CVE is genuine. Those checks require later semantic validation or trusted external data.

TASK 12B – EVALUATION OF THE ORIGINAL DESIGN (2 MARKS)

The original design should be evaluated against functional correctness, diagnostic quality and performance. It may correctly recognise common keywords, identifiers, integers, strings and operators, but it needs to be checked for threat-specific indicators and error recovery.

- Strength: regular-expression rules make common token recognition concise and repeatable.
- Strength: generated scanners are easier to maintain than separate hand-written scanners at each branch.
- Limitation: an IPv4 regular expression checks only the general shape unless octet values are validated separately.
- Limitation: generic identifier rules can conflict with special literals if rule order is not designed carefully.
- Limitation: stopping after one lexical error is unsuitable for batch validation because users need all errors in one run.
- Evaluation method: run a labelled input corpus and compare expected tokens and errors with actual scanner output.

No numerical success rate should be claimed without a documented test corpus and actual execution results.

TASK 12C – ADAPTATION / REDESIGN (2 MARKS)

Decision: MODIFY the existing scanner rather than redesign it completely. Flex already supports the required regular expressions, and the existing keyword, identifier, operator and integer rules can be retained. The following token definitions can be added:

```
SHA256     [A-Fa-f0-9]{64}
CVE        CVE-[0-9]{4}-[0-9]{4,}
URL        (https?|ftp)://[^ \t\r\n;\"<>]+

{URL}      { emit("URL", yytext); }
{CVE}      { emit("CVE", yytext); }
```

```
{SHA256}  { emit("SHA256_HASH", yytext); }
```

Place special literal rules before the general identifier and fallback rules. Flex selects the longest match; if two rules match the same number of characters, the earlier rule has priority. The fallback rule should report an error and consume the unexpected character so scanning can continue. IPv4 values should be checked by a helper function that parses and validates all four octets.

TASK 12D – EVIDENCE-BASED DEFENCE (2 MARKS)

The MODIFY decision should be defended by a controlled comparison. Prepare a corpus containing ordinary ThreatRule rules, valid hashes, CVE identifiers, URLs and malformed values. Run the original and modified scanners on exactly the same file, on the same computer and with the same compiler settings. Repeat the test and compare median scan time.

Measure	Original scanner	Modified scanner
Valid original token classes	Record actual count	Record actual count
HASH/CVE/URL literals correctly recognised	Record actual count	Record actual count
Lexical errors reported	Record actual count	Record actual count
Errors with line numbers	Record actual count	Record actual count
Valid tokens found after an error	Record actual count	Record actual count
Median scan time (seconds)	Record median	Record median

The modified scanner is expected to recognise the additional token classes and continue after malformed input. It may perform slightly more matching work because it contains additional patterns. The change should be accepted if measured results show better coverage and diagnostics without an unacceptable loss of performance. Fill the table with actual observations; do not enter estimated or invented measurements.

CONCLUSION

This assignment designed a compiler front-end for the ThreatRule domain-specific language used to share cyber-threat detection rules between branch SOCs and a central SOC. The main compiler phases, front-end/back-end grouping and compiler-construction tools were explained. Token classes and regular

expressions were specified, and finite-state transition diagrams illustrated how identifiers, integers, relational operators and IPv4 candidates are recognised.

A Flex-based scanner was presented with keyword recognition, identifiers, line tracking, comments, strings, operators and recoverable lexical errors. The design was extended to recognise SHA-256 hashes, CVE identifiers and URLs. The recommended approach is to modify and test the existing scanner, validate domain-specific values carefully, and use measured large-file performance and original-versus-modified comparisons before central deployment.

APPENDIX A – FLEX PROGRAM IMPLEMENTATION NOTES

The full standalone Flex specification is provided in Task 8. For a production-quality submission, the following improvements should be added before deployment:

- An IPv4 helper that splits the candidate on dots and checks exactly four octets, each between 0 and 255.
- A dedicated unterminated-string rule so an opening quote without a closing quote generates a clear error.
- A block-comment state if the language supports /* ... */ comments, including an error for an unclosed comment.
- Dynamic memory management or a hash-based symbol table if identifiers must be stored.
- Parser-generated token declarations when integrating with Bison, instead of local demonstration token values.
- Unit tests for boundary cases, including empty input, long identifiers, malformed URLs, invalid hashes and multiple errors on one line.

APPENDIX B – TEST DATA AND COMMANDS

B.1 Example ThreatRule file

```
RULE ssh_attack IF src_ip == 10.0.0.5 AND attempts > 5
THEN ALERT "Repeated login attempts";

RULE known_vulnerability IF cve == CVE-2024-12345
THEN ALERT "Review affected system";

// The next line intentionally contains an invalid character:
```

```
RULE bad_rule IF attempts @ 5 THEN BLOCK;
```

B.2 Commands

```
flex threatrule.l
gcc lex.yy.c -o threatrule
./threatrule < sample.tr
```

On Windows, run the generated executable from Command Prompt and use the same input-redirection syntax, for example: threatrule.exe < sample.tr. The exact command depends on the installed Flex/GCC distribution.

REFERENCES

7. Aho, A. V., Lam, M. S., Sethi, R., and Ullman, J. D., Compilers: Principles, Techniques, and Tools, 2nd Edition, Pearson, 2006.
8. Levine, J. R., Lex & Yacc, 2nd Edition, O'Reilly Media, 1992.
9. Flex, Fast Lexical Analyzer Generator, official manual and documentation.
10. GNU Bison, The GNU Bison Manual, official documentation.
11. National Institute of Standards and Technology (NIST), Secure Hash Standard (FIPS PUB 180-4), 2015.
12. MITRE, Common Vulnerabilities and Exposures (CVE) Program, official CVE identifier format and program information.